PPSITMAJM402 DEEP LEARNING

Ms. Sophia Dcruz

UNIT IV: DEEP LEARNING RESEARCH

INTRODUCTION

- Unsupervised learning lives in the quiet territory where data exists without labels, and yet structure must be discovered.
- In supervised learning, accuracy improves when we have large amounts of labeled data, but in reality such data is often unavailable, expensive, or unreliable.
- This leads us to explore models that can understand patterns, structure, and hidden relationships without being told what the correct output is.
- Unsupervised learning does not directly solve prediction problems like supervised learning, but it reveals the internal organization of data.
- One of the biggest challenges here is high dimensionality, which increases computational and statistical complexity.
- To handle this, several approaches such as linear factor models, autoencoders, and representation learning have been developed to uncover meaningful patterns from raw data.

LINEAR FACTOR MODELS

- Linear Factor Models assume that observed data is generated from a small number of hidden variables called latent variables.
- These latent variables combine linearly, along with some noise, to form the observed data.
- The model is expressed as $X=Wh+b+noise$, where h represents hidden factors, W is the weight matrix, and noise accounts for uncertainty.
- This idea helps explain how complex data can arise from simpler underlying causes.
- Models such as Probabilistic PCA, Factor Analysis, and ICA are variations of linear factor models with different assumptions about noise and the distribution of latent variables.

Probabilistic PCA (PPCA)

- PPCA extends traditional PCA by giving it a probabilistic interpretation.
- PCA finds directions of maximum variance to reduce dimensionality, but PPCA assumes that data is generated from Gaussian latent variables with Gaussian noise of equal variance in all directions.
- As the noise approaches zero, PPCA becomes classical PCA.
- PPCA provides a statistical foundation to PCA and explains how most variations in data are captured by latent variables.

Factor Analysis

- Factor Analysis is similar to PPCA but assumes that each variable has its own noise variance.
- Instead of equal noise in all directions, Factor Analysis uses a diagonal covariance matrix with different variances.
- It focuses on identifying independent explanatory factors that influence observed variables.
- This makes it useful in domains like psychology and finance where variables have different uncertainty levels.

Independent Component Analysis (ICA)

- ICA separates mixed signals into independent underlying signals.
- It assumes that latent variables are independent and non-Gaussian.
- ICA is widely used in signal processing, such as separating multiple voices recorded by different microphones.
- By maximizing independence, ICA recovers original source signals from observed mixtures.

Slow Feature Analysis (SFA)

- SFA is based on the principle that meaningful features change slowly over time even when raw data changes quickly.
- For example, in a video, pixel values change rapidly, but the presence of an object changes slowly.
- SFA extracts these slowly varying features from time-based signals, making it useful in video processing and motion tracking.

Sparse Coding

- Sparse Coding assumes that only a few latent variables are active at any time.
- This sparsity leads to efficient representation of data.
- The model still follows the linear factor structure but enforces that most values in the hidden representation are zero.
- Sparse coding is widely used in image processing and feature learning.

Manifold Interpretation of PCA

- The Manifold Interpretation of PCA explains that high-dimensional data often lies on a lower-dimensional curved surface called a manifold.
- PCA tries to approximate this manifold with a linear surface, which works only when relationships are linear.
- For nonlinear data, manifold learning techniques are required.

Autoencoders

- Autoencoders are neural networks trained to reproduce their input at the output.
- They consist of an encoder, a code (latent representation), and a decoder.
- The encoder compresses the input into a smaller representation, and the decoder reconstructs the input from this code.
- Since no labels are required, autoencoders are self-supervised.
- They are widely used for dimensionality reduction, feature extraction, and denoising.
- They are Data-specific.
- Autoencoders are lossy they don’t yield the exact same ouput as the input, it can be a degraded representation.
- To train an autoencoder we need the raw input data.

TYPES OF AUTOENCODERS

1. **Undercomplete Autoencoders** have a code dimension smaller than the input dimension. This forces the network to learn the most important features of the data instead of copying the input directly.
2. **Regularized Autoencoders** add constraints such as sparsity, smoothness, or robustness instead of reducing code size. These constraints force the model to learn meaningful features even when the code dimension is large.
3. **Denoising Autoencoders** add noise to the input and train the model to reconstruct the original input. This teaches the model to remove noise and learn robust features.

APPLICATIONS OF AUTOENCODERS

- Autoencoders have been effectively used in dimensionality reduction and information retrieval tasks.
- Autoencoders are good at denoising of images.
- Autoencoders are used in Feature Extraction, they help to learn important hidden features of the input data.
- Variational Autoencoder (VAE), is used to generate images.
- Autoencoders are used in Sequence-to-Sequence Prediction.
- Deep Autoencoders can be used in recommending movies, books, or other items.

Representation Learning

- Representation Learning focuses on transforming data into forms that make learning easier.
- A good representation simplifies tasks like classification or prediction.
- Hidden layers in neural networks learn such representations.
- Using unlabeled data to learn representations helps reduce overfitting when labeled data is scarce.
- Example:
- 670 ÷ 6 in decimal → easy
- In Roman numerals → nightmare.

CHAPTER 2: APPROXIMATE INFERENCE

INTRODUCTION

- Approximate inference methods are used to learn realistic models from large datasets such as images, videos, and text, while reducing computation time and maintaining acceptable accuracy.
- In Bayesian learning, inference ideally requires computing the posterior probability of model parameters.
- However, this posterior is often very difficult or impossible to compute exactly due to complex integrations and high dimensionality.
- Therefore, we replace the true posterior with a simpler, biased but tractable approximation and perform inference using this approximation.

Posterior Probability

- Posterior probability is the updated probability of an event after considering new evidence.
- It is calculated using Bayes' theorem by combining prior belief with observed data.

The formula to calculate a posterior probability of A occurring given that B occurred:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} = \frac{P(A) \times P(A)}{P(B)}$$

- Here, prior probability represents initial belief, and posterior probability reflects updated belief after observing data.
- Prior probability represents what is originally believed before new evidence is introduced, and posterior probability takes this new information into account.
-

Approximate Inference in Machine Learning

- Learning can be seen as an inference problem where we want the posterior distribution of parameters $p(\theta|D)$.
- However, computing $p(D)$ is often intractable.
- Approximate inference solves this by either sampling from the posterior (stochastic methods) or turning inference into an optimization problem (deterministic methods).
- Stochastic methods use Monte Carlo sampling (e.g., MCMC). Deterministic methods use variational optimization.

You have:

A model MMM

Some parameters θ inside that model

Some data D={xi}

Learning means: “Given the data, what values of parameters θ make sense?”

In probability terms, this is: $p(\theta \mid D, M)$

This reads as: Probability of parameters after seeing the data.

This is called the posterior.

Bayes politely tells us: $$p(\theta \mid D, M) = \frac{p(D \mid \theta, M)\, p(\theta \mid M)}{p(D \mid M)}$$

Term	Meaning
$p(D \mid \theta)$	Likelihood — how well parameters explain data
$p(\theta)$	Prior — what we believed before data
$p(D)$	Evidence — nightmare term

That last one, $p(D)$, is where dreams go to die.

That denominator: $$p(D) = \int p(D \mid \theta)\, p(\theta)\, d\theta$$

This integral is over all possible parameter values.

In deep learning models, this means integrating over millions of dimensions.

This is mathematically… adorable. Computationally… impossible.

So exact Bayesian inference? Not happening. We cheat. Elegantly. This is called Approximate Inference.

We cannot compute the posterior.

So we approximate it.

Two philosophies:

Method	Idea	Style
Stochastic	Sample from the posterior	Randomness, Monte Carlo
Deterministic	Optimize a simpler distribution to mimic posterior	Optimization

1. Stochastic Method — Monte Carlo (Sampling)

Key idea: If you can't compute the distribution, just sample from it.

Instead of computing expectation exactly, average over samples.

Example intuition

Imagine you want the average height of people in Mumbai.

You don't measure everyone.

You randomly sample 1000 people and average.

We want properties of $p(\theta \mid D)$

We draw many samples of θ from it.

This is done using **Markov Chain Monte Carlo (MCMC)**.

Even when posterior is ugly, MCMC can still *walk around it* and sample.

2. Deterministic Method — Variational Inference

"Posterior is too ugly. Let's fake it with a simpler, nice distribution."

We choose a simple distribution $q(\theta)$ (like Gaussian).

Then we **optimize** its parameters so that: $q(\theta) \approx p(\theta \mid D)$

We do this by minimizing a distance between them (KL divergence).

So instead of sampling.. We **train** a distribution.

This becomes an optimization problem, like neural network training.

Approximate Inference in Deep Learning

- In deep learning, models contain visible variables v and latent variables h.
- Computing $p(h|v)$ becomes very difficult due to interactions among latent variables in graphical models.
- These interactions make exact inference exponentially expensive, even for models with one hidden layer like sparse coding.

Inference as Optimization (ELBO)

- To handle intractability, inference is converted into an optimization problem using the Evidence Lower Bound (ELBO).
- $L(v, \theta, q) = \log p(v; \theta) - D_{KL}\,(q(h \mid v) \,||\, p(h \mid v; \theta))$
- ELBO is always less than or equal to the true log-likelihood.
- When q=p(h|v), ELBO becomes exact.
- By choosing a tractable q, we optimize this bound instead of the true probability.

Expectation Maximization

- EM is used for maximum likelihood estimation when latent variables are present.
- It alternates between estimating latent variables (E-step) and optimizing parameters (M-step) until convergence.
- E-Step: Estimate hidden variables
- M-Step: Maximize/ Update parameters
- The EM algorithm is useful for discovering the values of latent variables.
- t can be used to fill the missing data in a sample.
- It can be used as the basis of unsupervised learning of clusters.
- The E-step and M-step are often pretty easy for many problems in terms of implementation.
- It is always guaranteed that likelihood will increase with every iteration.
- EM algorithm has slow convergence.
- It makes convergence to the local optima only.

Maximum A Posteriori (MAP)

- Instead of computing full distribution $p(h|v)$, MAP finds the most likely value of h.
- $h^* = \mathit{argmax}\ p(h|v)$
- Using Bayes rule: $P(A \mid B) = (P(B \mid A) * P(A)) / P(B)$
- MAP is a Bayesian alternative to Maximum Likelihood Estimation.

Variational Inference and Learning

Variational inference maximizes ELBO over a restricted family of distributions

q is assumed to factorize:

$$q(h|v) = \prod_i q(h_i|v)$$

This is called the mean-field approach. Structured variational inference allows more flexible dependencies.

Discrete Latent Variables

- For discrete latent variables, q can be represented using probability tables.
- Optimization is done using fixed-point equations or gradient methods.
- Since this is inside the learning loop, it must be fast.

CHAPTER 3: DEEP GENERATIVE MODELS

Introduction to Generative Models

- A generative model learns the distribution of the data itself. Instead of predicting labels, it learns: “How likely is this example to exist?”
- For example, a model that predicts the next word in a sentence is generative because it assigns probability to sequences.
- Applications include generating faces, repairing damaged photos, converting day to night images, creating emojis, cartoons, 3D objects, predicting video frames, and much more.
- These are not classification problems. These are creation problems.

Deep Generative Models (DGM)

- DGMs are deep neural networks trained to approximate very complex probability distributions using samples.
- The goal is twofold:
  - Estimate likelihood of data
  - Generate new samples similar to training data
- However, training DGMs is mathematically difficult because:
  - True data distribution is unknown
  - Generator inversion is hard
  - Latent space dimension is unknown and chosen as hyperparameter
- Example: Train on 10,000 celebrity faces → model generates a new face that never existed but looks real.

Supervised vs Unsupervised Learning

- Supervised learning learns mapping: X→y.
- Unsupervised learning finds patterns only in X
- Supervised: Classification, Regression
- Unsupervised: Clustering, Generative modeling
- GANs fall in unsupervised learning, yet their training looks like supervised learning because of discriminator feedback.

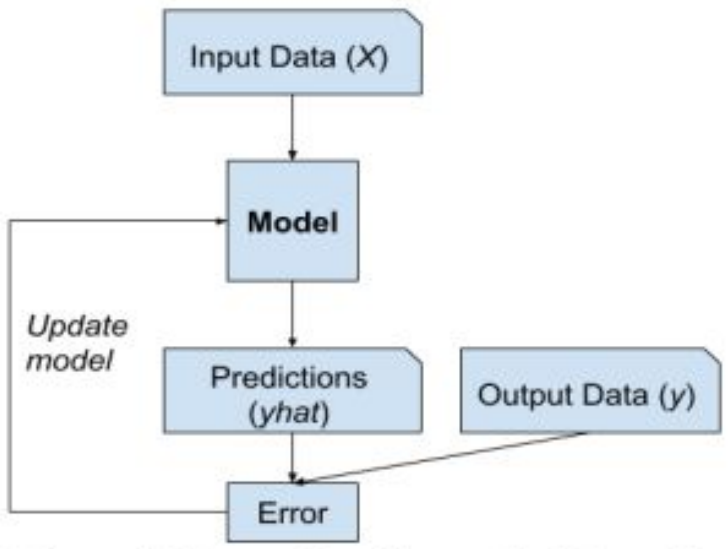


Figure 9.1 Example of Supervised Learning

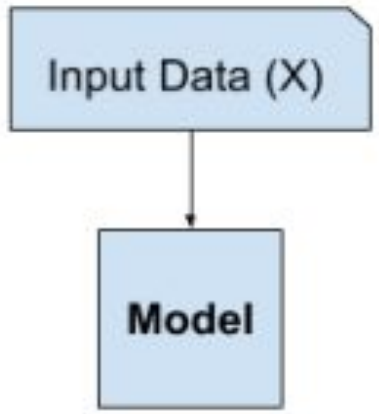
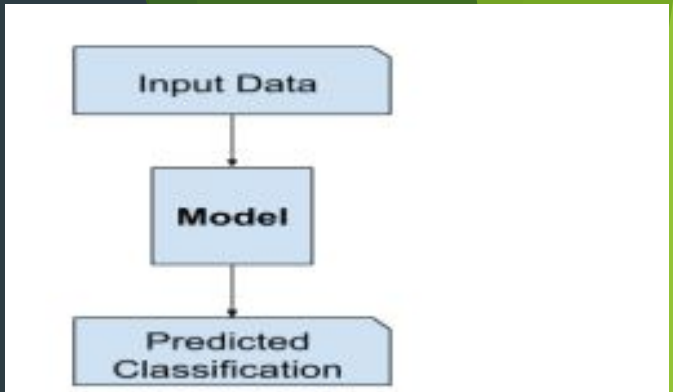


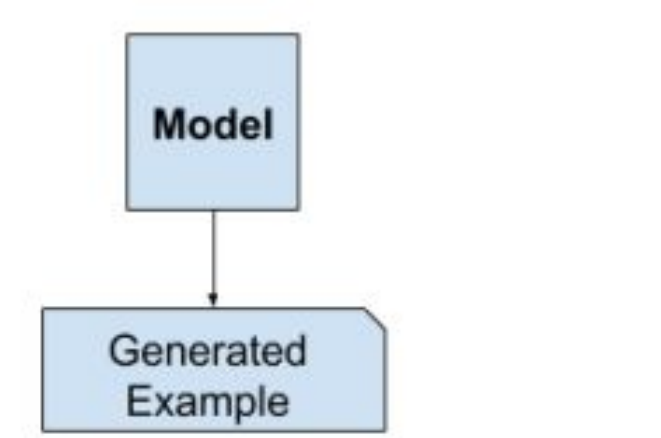
Figure 9.2 Unsupervised Learning

Discriminative vs Generative Modeling



Example of Discriminative Modeling

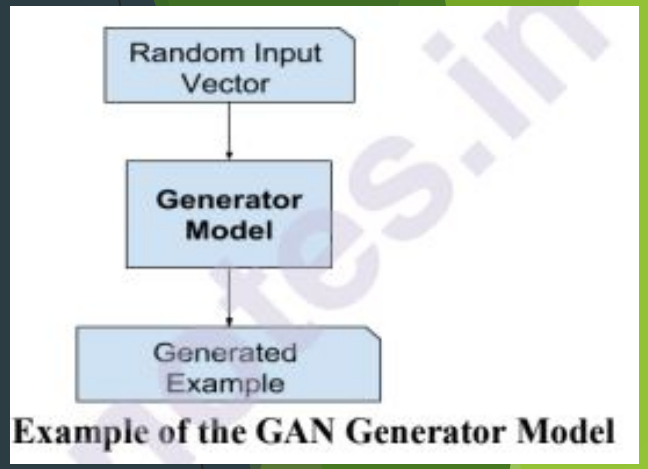
- Discriminative models learn boundary between classes.
- Generative models learn how data is distributed.
- Naive Bayes, GMM, LDA, RBM, DBN, VAE, GAN — all generative.
- A good generative model can produce samples indistinguishable from real data.



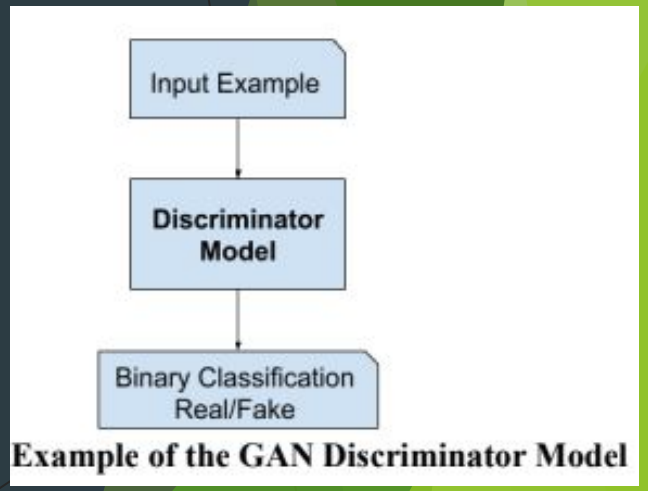
Example of Generative Modeling

Generative Adversarial Networks (GANs)

- GANs consist of two networks:
- Generator: creates fake samples from random noise
- Discriminator: identifies real vs fake
- The generator takes a random vector from a latent space and generates data.
- After training, only the generator is used.
- Example: Random vector → realistic human face.



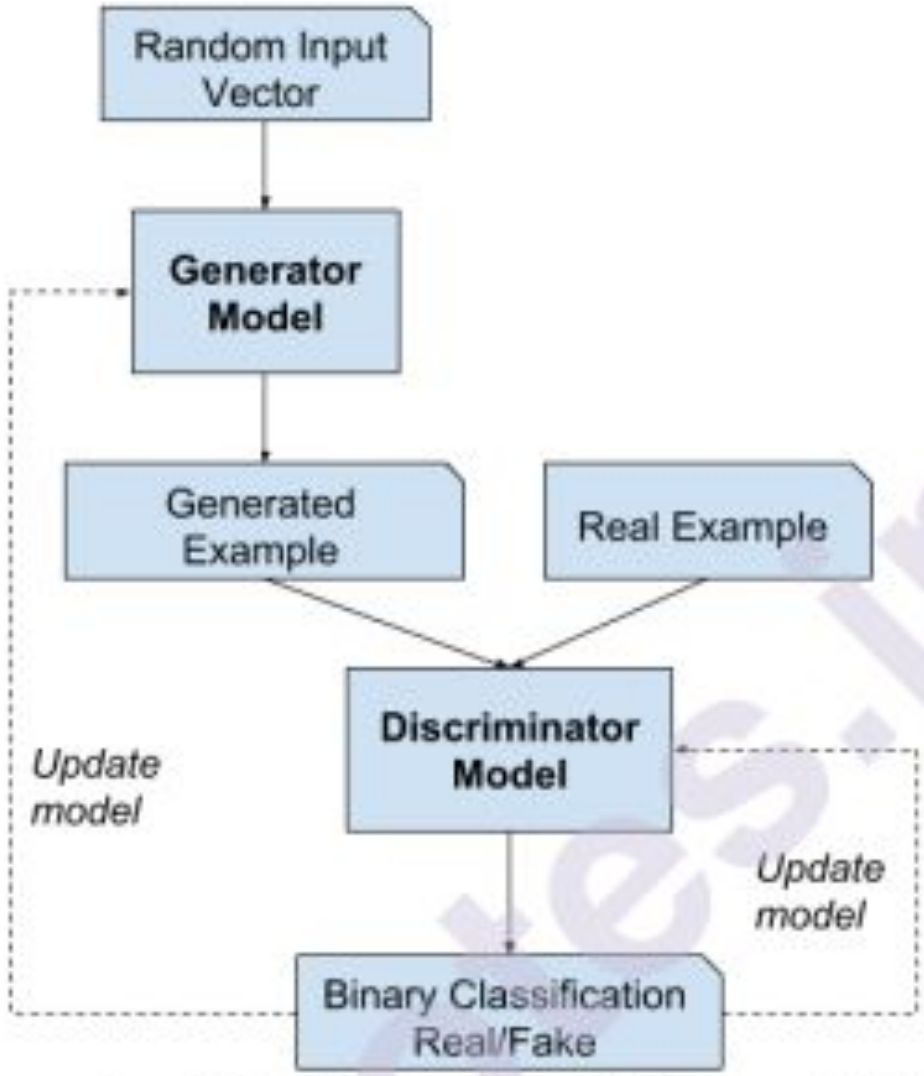
Example of the GAN Generator Model



Example of the GAN Discriminator Model

GANs as a Two Player Game

- GAN training is a competition:
  - Generator tries to fool discriminator
  - Discriminator tries to detect fakes
- This is a zero-sum game.
- At ideal equilibrium: Discriminator outputs 50% for everything because fake = real.
- This adversarial training is why GANs are powerful.



Example of the Generative Adversarial Network Model Architecture

GANs with CNNs

- GANs mostly use CNNs for image tasks because:
- Images are high dimensional
- CNNs capture spatial features
- Output is visually verifiable

Conditional GANs

- Conditional GANs generate specific types of output.
- Input: noise + condition (label/image/text)
- Examples:
  - Generate only female faces
  - Convert day image to night image
  - Text → Image
  - Image → Image translation
- This enables style transfer, colorization, super resolution, art generation.

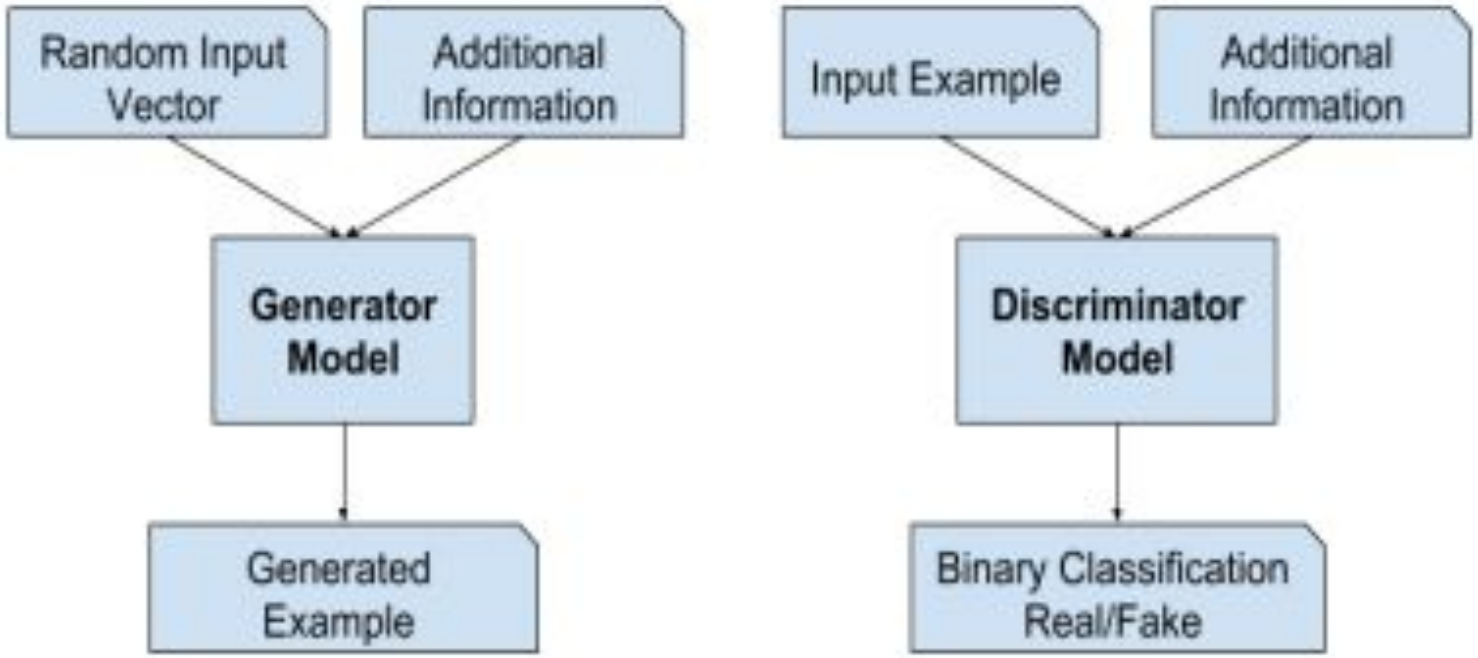


Figure 9.8 Example of a Conditional Generative Adversarial Network Model Architecture

Boltzmann Machines

- Boltzmann Machines are stochastic, undirected networks where every node connects to every other.
- They use energy-based modeling.
- Probability of a state depends on its energy:

  Lower energy = higher probability.

- They learn system behavior in normal conditions to detect abnormalities.

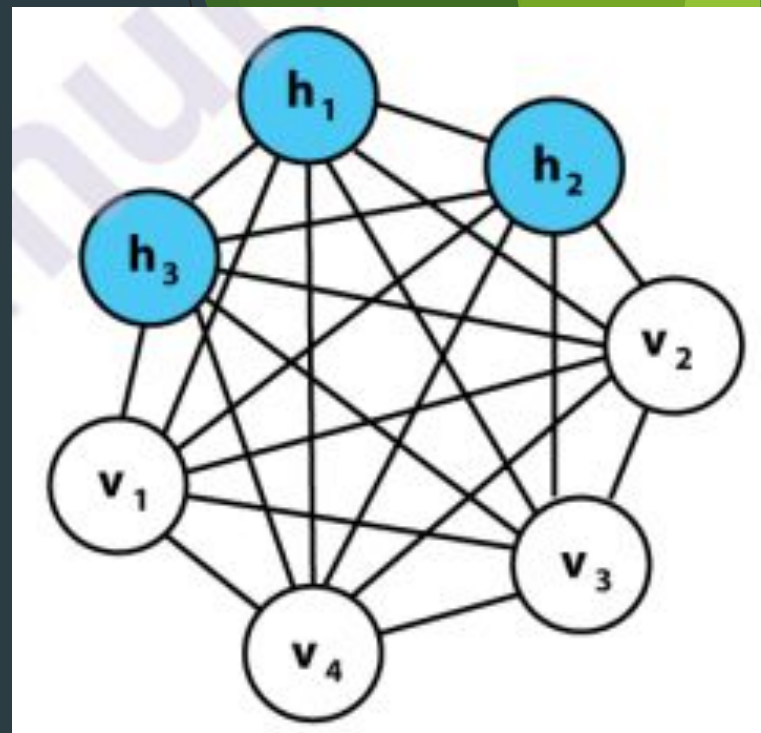


Figure 9.9 Boltzmann Machine

Restricted Boltzmann Machines (RBM)

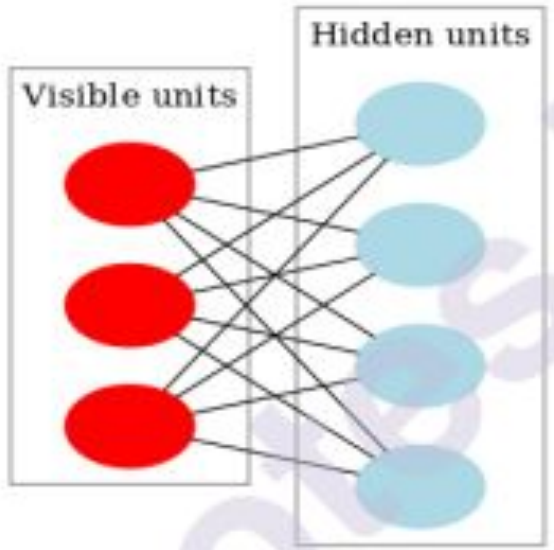


Figure 9.10 Boltzmann Machine units structure

- RBMs simplify Boltzmann Machines:
  - No visible-visible connections
  - No hidden-hidden connections
  - Only visible ↔ hidden
- Training occurs in two phases:
- Forward pass (visible → hidden)
- Reconstruction (hidden → visible)
- Goal: minimize reconstruction error: v(0)−v(1)
- Used in handwritten digit recognition and recommender systems.

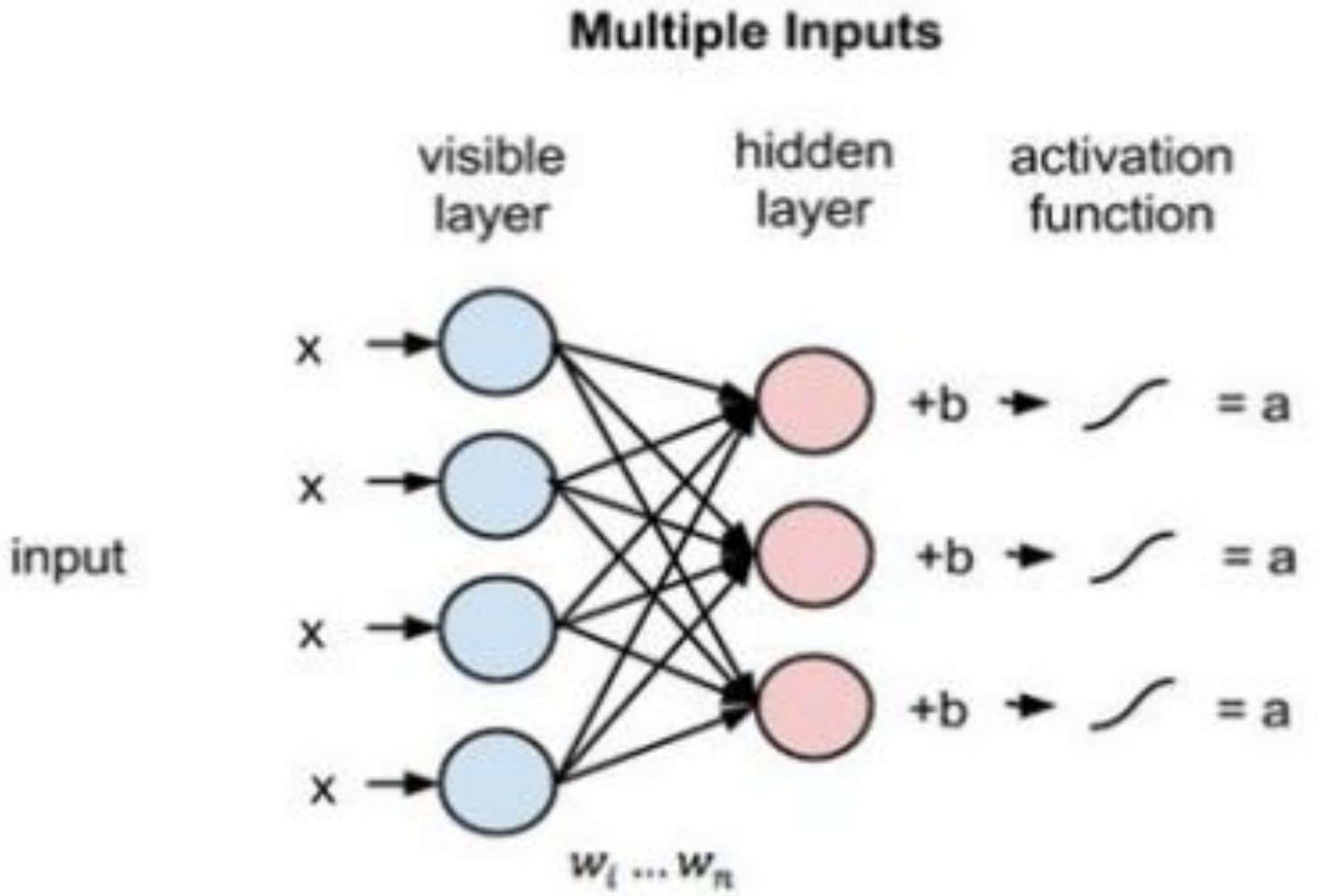


Figure 9.11 working of Restricted Boltzmann Machine

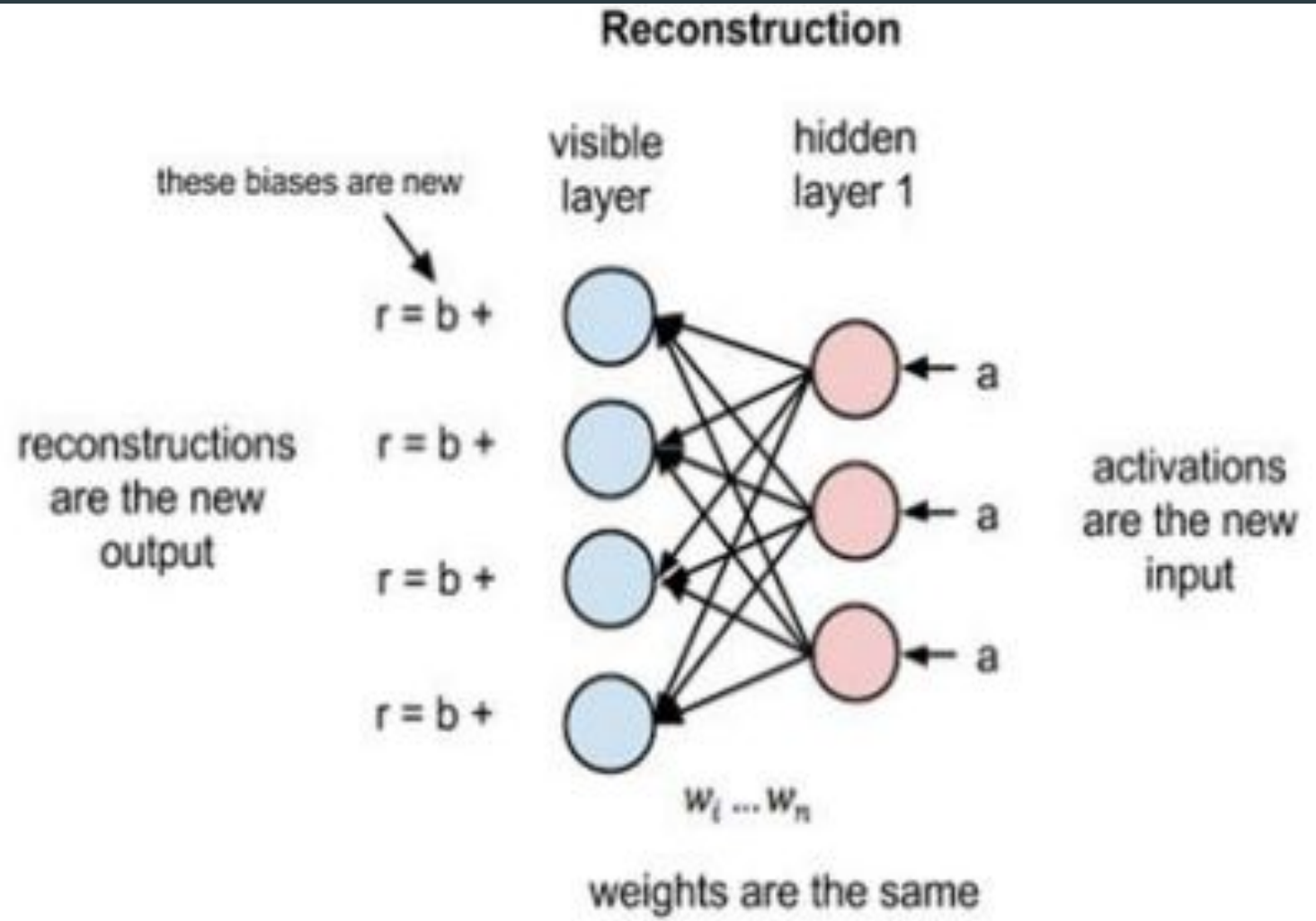


Figure 9.12 reconstructions in Restricted Boltzmann Machine

RBM vs Autoencoder

- Autoencoder: deterministic, minimizes reconstruction error.
- RBM: stochastic, models probability distribution using energy.
- RBM has two biases and probabilistic activations.

Deep Belief Networks (DBN)

- DBN = Stack of RBMs.
- Training occurs in two phases:

1. Pre-train Phase: Train each RBM layer greedily.
2. Fine-tune Phase: Use backpropagation like feedforward network.

- Algorithm:

1. Train first RBM on inpuT
2. Use hidden layer output as input for next RBM
3. Repeat for layers

- Weight Update in DBN
- Two phases:

1. Positive Phase: Compute probability of hidden given visible
2. Negative Phase: Reconstruct visible from hidden

- Weight update depends on difference between these probabilities.
- Advantages:
  - Avoid vanishing gradients
  - Efficient layer-wise learning
  - Later replaced by autoencoders and ReLU networks.

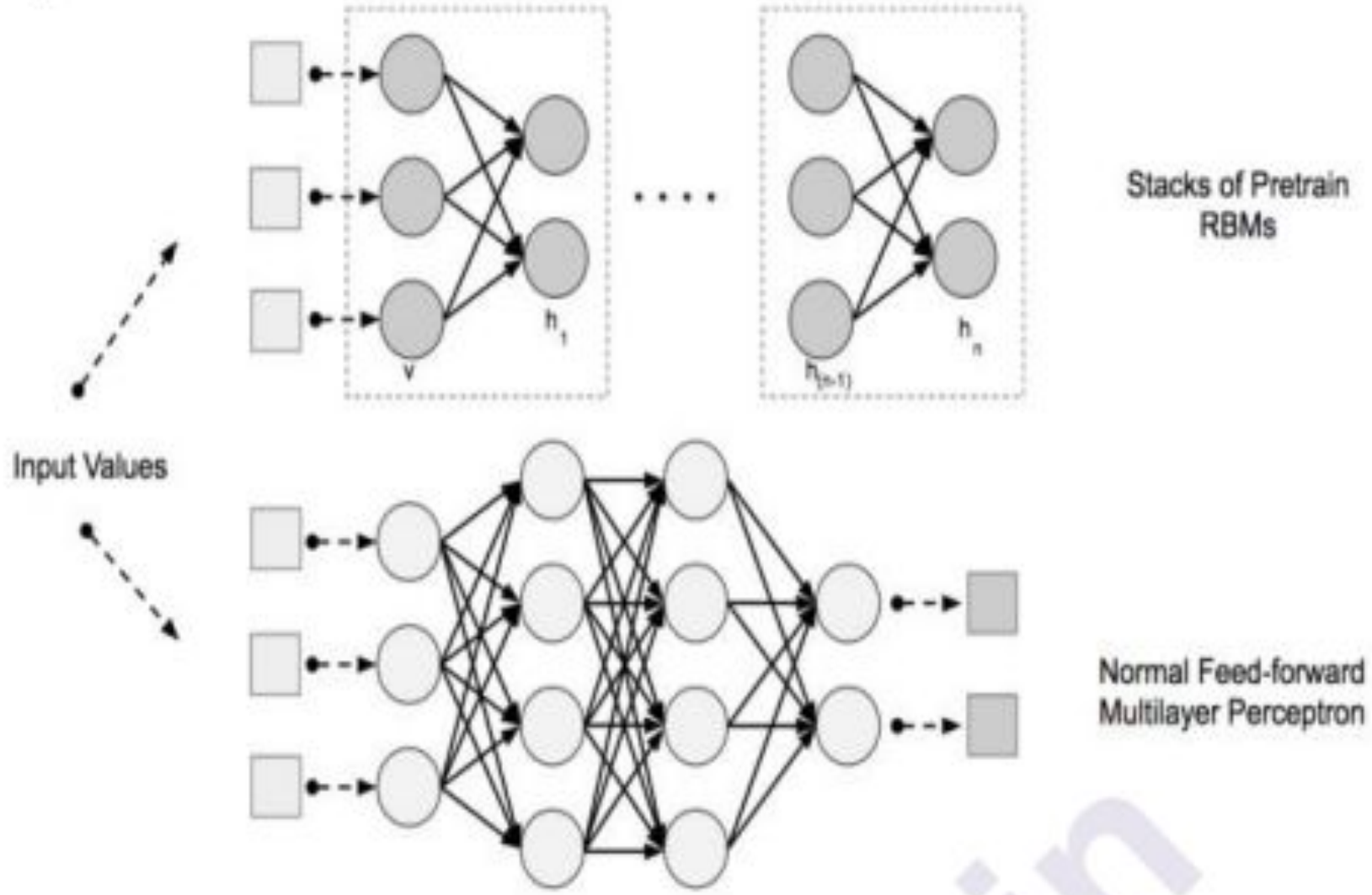


Figure 9.13 Deep Belief Networks